

Balimmo Landing Page — Written Walkthrough

by **Matthew Raymond Hartono**

Front-End / UI-UX Integrator take-home test · July 2026 · Repository: github.com/matthewraymondh/balimmo-landing

This is the walkthrough deliverable, submitted in written form as allowed by the brief's checklist. The full ranked audit (24 findings), scoring rubric, and measurement plan are in AUDIT.md in the repository — this document explains how I reasoned my way through the exercise.

How I framed the problem

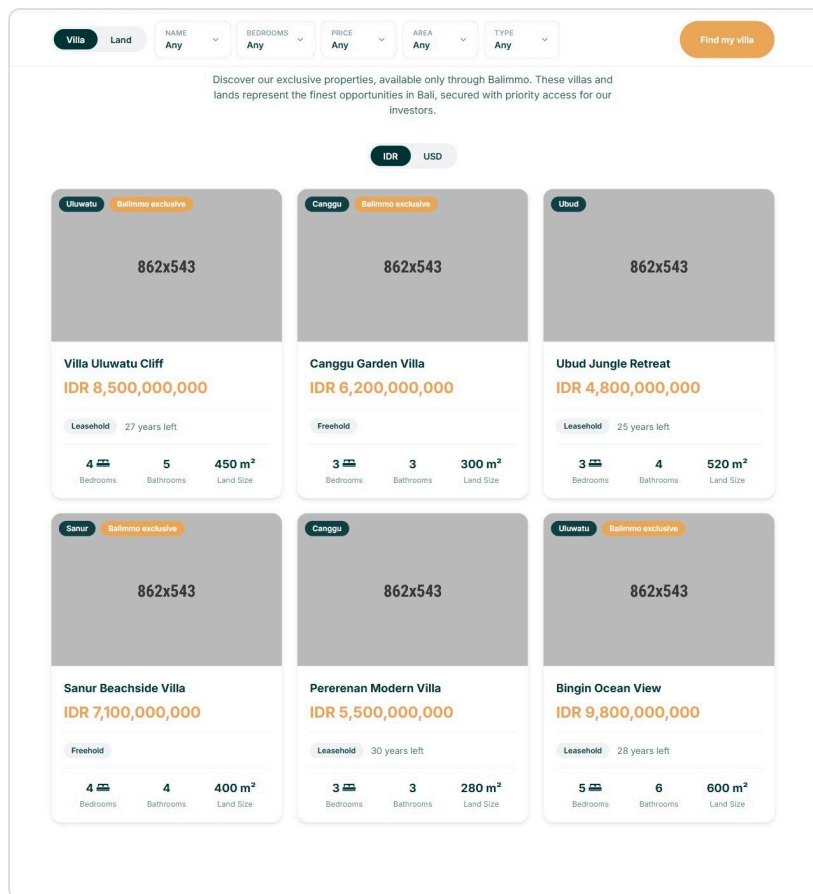
A real-estate landing page has exactly one job: turn a visitor into a lead — a search, a form submit, or a WhatsApp chat. So instead of auditing "is this page pretty," I audited it as a funnel: where does intent enter, and where does it leak out. Every problem I found is ranked by how directly it blocks that funnel, for the user and for the business.

On that lens the baseline score is **41/100** and my post-fix estimate is **65/100**. The remaining gap is deliberately not frontend polish — it's content and integration work (real listing data, a wired search flow) that no landing-page code can substitute for.

Assumptions I made (and how I checked them)

1. **The audited artifact is a static clone.** The repo documents that its mock data stands in for the Laravel/Livewire original's dynamic listings, and the tunnel URL serves this exact repo via `vite dev`. I verified the production site (balimmo.fr) separately — it has real, optimized photography. So findings like "all listings show placeholders" are framed as *what the data integration must deliver*, not as accusations that the business lacks photos.
2. **No analytics were provided**, so both scores are reasoned estimates from heuristics — the audit says this explicitly, and the measurement plan in AUDIT.md §5 is how I'd replace estimates with data.
3. **Auditing a local production build is representative** — and strictly generous to the current page, since the public URL actually serves unminified dev-server output.

The most critical finding — and why I deliberately didn't "fix" it

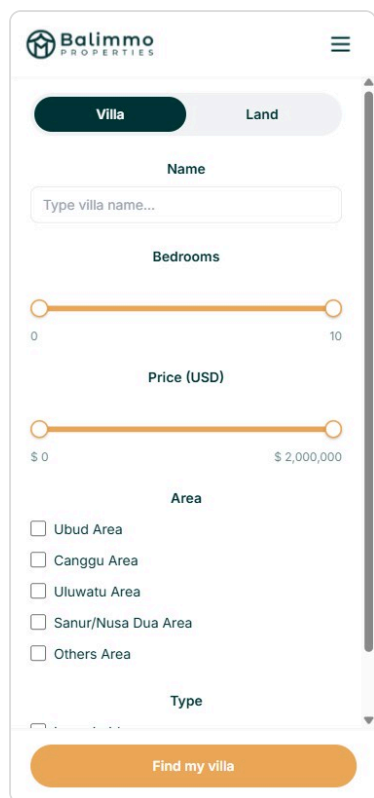


The original listings section: six cards, all showing the same grey 862×543 placeholder.

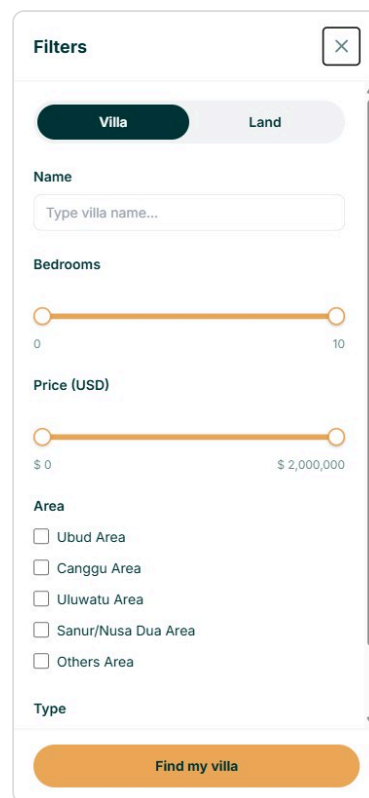
All six "exclusive listings" render the same grey 862×543 placeholder — I checked the files, and the three thumbnails are byte-for-byte identical (same MD5). A buyer scrolling past six identical grey boxes doesn't think "placeholder"; they think the site is broken, or the listings aren't real. Photos are the #1 decision input for property buyers, so this caps the money section's conversion at roughly zero.

I did *not* paper over it with stock images, because that would misrepresent listings. The honest fix is upstream: the listing-image integration is the top open dependency, those images must come through an optimization pipeline (resized, AVIF/WebP, `srcset` — the 477–746 KB PNGs elsewhere in the repo suggest no pipeline exists yet), and there must be a designed fallback so a raw grey box is never an empty state users see. Everything I *did* fix was chosen to still hold up once that integration lands.

Fix 1 — the mobile search trap



Before: the sheet opens *under* the navbar — no header, no close button, hamburger still showing.



After: the sheet overlays the navbar with a visible "Filters" header and working close button.

On a phone, tapping **Search** opens the filter sheet — but its "Filters" header and close button render *behind* the sticky navbar (left screenshot: no header, undimmed navbar showing through). I verified with `document.elementFromPoint` that tapping where the X should be actually hits the hamburger menu. And since the sheet is full-width on a 375px phone, there's no "outside" to tap either: the user is trapped.

The root cause is a classic CSS stacking-context bug: the hero's `relative z-10` wrapper caps everything inside it, so the sheet's `z-[60]` loses to the navbar's `z-50`. The fix portals the sheet to `document.body`, escaping the stacking context entirely, and adds real dialog semantics: `role="dialog"` + `aria-modal`, focus moves to the close button on open and back to the trigger on close, and the overlay tap closes it. Mobile is typically the majority of property-portal traffic — this bug sat at the very top of that funnel.

Fix 2 — the eight-megabyte hero

The original loads an 8.2 MB autoplay background video on the critical path for every visitor — mobile data included — plus ~2.8 MB of eager below-fold PNGs, with no reduced-motion handling and no pause control.

The fix keeps the video but takes it off the critical path: the poster image paints immediately (preloaded, `fetchpriority="high"`), the video element only mounts after the `window.load` event, and it never loads at all for `prefers-reduced-motion` or Data-Saver/2G users. A pause/play control satisfies WCAG 2.2.2. Every below-fold image is now lazy-loaded with explicit dimensions (CLS stays 0.00), and I dropped a font family that was loaded but never used.

Here is what actually goes over the wire, measured on cold production builds of both versions (fresh browser context, mobile viewport, Fast 4G + 4× CPU emulation, no scrolling — collected via the Resource Timing API):

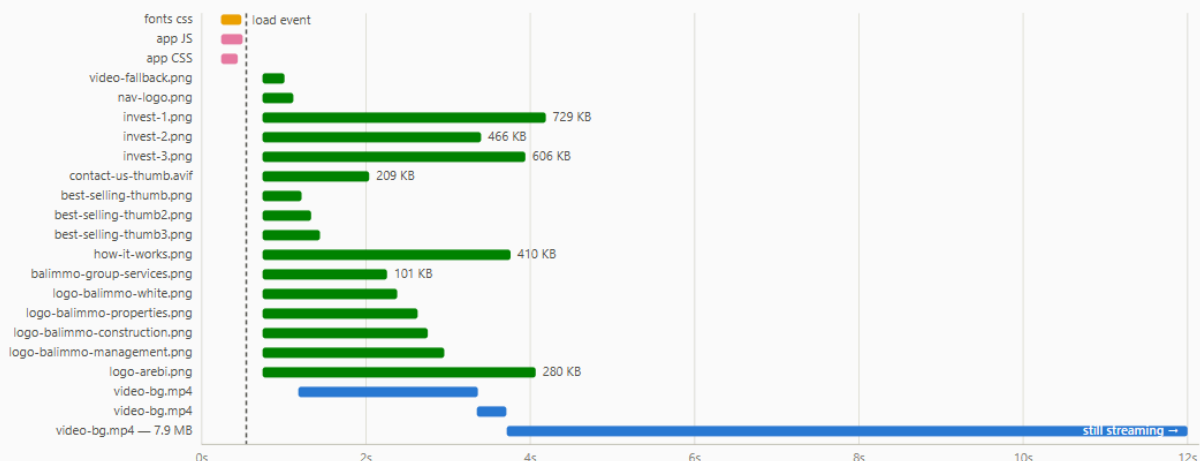
Fix 2 — what actually goes over the network on first load

Production builds of both versions, measured cold (fresh browser context, cache empty), mobile viewport, Fast 4G + 4× CPU throttling, no scrolling. Each bar is one request from start to finish.

■ hero video ■ images ■ app JS / CSS ■ fonts

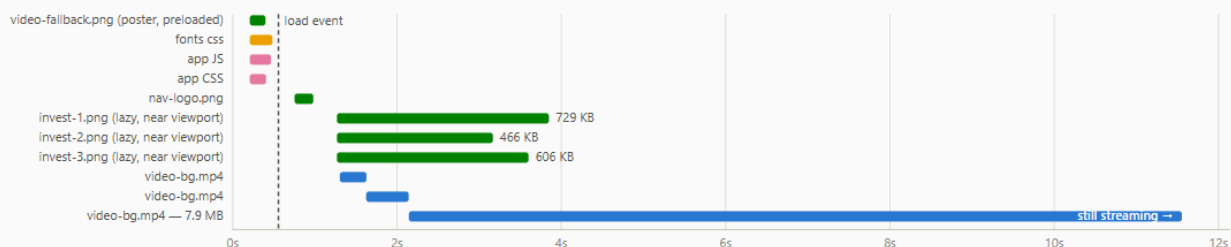
BEFORE — original page

All 16 images (2.9 MB) download immediately whether or not the user ever scrolls, competing with the 8 MB video from $t \approx 1.2$ s.



AFTER — with the fix

The page settles at ~ 0.6 s after ~ 180 KB. The video waits for the load event; only the 3 images nearest the viewport load (the other 13 wait for scroll). Reduced-motion / Data-Saver users never fetch the video at all (~ 8 MB).



Measured via the Chrome DevTools Protocol (Resource Timing API), July 2026. Time axis clamped at 12 s. On capable connections both versions eventually fetch the video — the fix moves it behind the load event and drops it entirely for prefers-reduced-motion, Data-Saver and 2G users; below-fold images load on demand instead of all at once.

Measured network waterfall: before, all 16 images download at once while the 8 MB video competes; after, the page settles on ~ 180 KB and everything heavy is deferred or conditional.

Two things to be precise about, because they matter for honest interpretation. First, on a capable connection the video still downloads *eventually* in the after-version — by design; desktop users keep the ambience. The win is **prioritization**: everything the user needs settles in ~ 180 KB at ~ 0.6 s, and the 8 MB stops competing with it. Second, the total saved is **conditional**: 13 of the 16 images (~ 1.1 MB) now wait until the user actually scrolls to them, and reduced-motion, Data-Saver and 2G users never fetch the video at all — an 8 MB saving for exactly the audiences most hurt by it. On a fast dev machine the two versions look identical, which is precisely the point: the users this fix serves are the ones on hotel Wi-Fi and mobile data, who are also the ones most likely to bounce.

Fix 3 — the lead form and the buttons

The 'Before' screenshot shows a form with fields for First name, Last name, Email, Phone, and Message. The 'After' screenshot shows the same form with red error messages: 'Please enter your first name.', 'Please enter your email so we can reply to you.', and 'Tell us a little about what you're looking for.' The 'After' confirmation screen shows a 'Thank you, Sarah!' message and a 'Talk to us now on WhatsApp' button.

Before — taken right *after* clicking submit on an empty form: no reaction at all.

After — empty submit: highlighted fields with inline error messages.

The form the whole page funnels into was a black hole: no validation, no error or success state, labels not programmatically associated, no `name / autocomplete` (which breaks browser autofill), and a submit that silently did nothing.

Rebuilt: associated labels, inline validation with `role="alert"` errors and `aria-invalid`, focus jumping to the first invalid field, and a clear confirmation state after submit — with a WhatsApp fast-lane on it, because in the Bali market WhatsApp is *the* contact channel and the page didn't offer it anywhere above the footer.

Alongside it, contrast: white text on the brand gold is roughly **2:1** — a WCAG failure on every primary CTA and every price. It's dark teal on gold now (~6.5:1) — compare the "Send my request" button across the before/after screenshots above — with a darker gold token for prices on white. Lighthouse (mobile, production build): **Accessibility 96 → 100, SEO 92 → 100** — the SEO delta also reflects the added Open Graph/Twitter meta and robots.txt.

What I'd measure to prove it worked

None of this is proven until it's measured, and the page ships with zero analytics. First change after these fixes: event instrumentation — search submits with their filter payloads, the filter sheet's open→apply rate (a step change is expected from the trap fix), WhatsApp clicks, form errors per field, and real-user web-vitals. The primary KPI is **lead rate**: (form submits + WhatsApp clicks) / sessions. The A/B queue starts with moving the listings above the region sections on mobile, and with testing whether the hero video earns its bytes at all against a static image. The full event table, KPI targets, and test list are in AUDIT.md §5.